

Bitácora de Desarrollo — Iteración 4

Soporte CSS para la vista Carrito Extendida

Asignatura: DSY1104 — Desarrollo Web Frontend

Proyecto: Tienda online temática Pokémon — Edición Perla

Documento: Bitácora de Desarrollo — Capa de Presentación, iteración por vista

Versión del documento: 3.0

Fecha: 2025

Responsable técnico: Desarrollador Fullstack contratado

Cliente: Equipo de desarrollo DSY1104 (Gabriel Flores, Daniel Rangel, Hernaldo Silva)

Estado: Aprobado por el cliente — listo para commit y verificación final

1. Objetivo de la bitácora

Dejar constancia formal, trazable y verificable de las actividades realizadas sobre la **capa de presentación (CSS3)** del proyecto PokéTienda Pearl durante la **cuarta iteración de trabajo**, motivada por la reescritura del archivo `carrito.html` por parte de un segundo desarrollador especializado en HTML.

Esta bitácora constituye:

- Evidencia de avance para la **Evaluación Parcial 1** (indicadores IE1.1.2 y IE1.1.4).
 - Documento de traspaso entre el responsable técnico y el equipo de desarrollo.
 - Registro histórico de las decisiones de diseño y de los hallazgos estructurales detectados.
-

2. Contexto y antecedentes

Tras el cierre satisfactorio de la iteración anterior (Estilizado Detallado por Vista), el cliente solicitó una **segunda opinión técnica** sobre la vista `carrito.html`. El objetivo era validar la semántica del marcado, la accesibilidad y la preparación para la futura capa JavaScript.

Un desarrollador HTML externo reescribió el archivo, elevando considerablemente la calidad de la estructura. La versión resultante:

- Introduce un sistema BEM más estricto para el breadcrumb.
- Enriquece cada ítem del carrito con metadatos (SKU, estado, descripción, precio unitario, subtotal).

- Incorpora un **sistema completo de cupones** con feedback visual.
- Añade un **estado vacío** semánticamente correcto.
- Suma **sellos de confianza** y **productos sugeridos**.
- Prepara exhaustivamente los ganchos (`data-*`, `id`) para la capa JavaScript.

Sin embargo, este nuevo marcado introdujo **clases y modificadores no contemplados** por el `styles.css` vigente, además de un **problema estructural de layout** en el grid del carrito. La presente bitácora documenta el diagnóstico y la solución implementada.

3. Resumen ejecutivo de la iteración

La iteración consistió en **un solo ciclo de análisis, diagnóstico y corrección**, cuyo resultado es un bloque CSS adicional de aproximadamente 380 líneas, retrocompatible con la iteración anterior y sin necesidad de modificar el HTML.

Etapa	Acción	Resultado
Auditoría	Comparación del nuevo <code>carrito.html</code> con el CSS vigente	16 selectores nuevos sin cobertura
Diagnóstico	Análisis del layout del grid del carrito	Descolocamiento detectado con <code>carrito-vacio</code> oculto
Corrección	Bloque CSS de soporte + <code>grid-template-areas</code>	Cobertura 100 % del nuevo HTML

4. Actividades realizadas

4.1 Auditoría del nuevo marcado

Se comparó el `carrito.html` recibido contra el `styles.css` vigente y se identificaron las clases, modificadores y elementos sin cobertura de estilos. La tabla siguiente documenta los 16 hallazgos clasificados por grupo funcional:

#	Clase / Selector nuevo	Función	Cobertura previa
1	<code>.breadcrumb__lista / __item / __enlace / __item--actual</code>	Breadcrumb con BEM	⚠ Parcial
2	<code>.carrito-view__cabecera / __subtitulo</code>	Encabezado del carrito	✗

3	<code>.carrito-item__enlace</code>	Wrapper <code><a></code> de la imagen	✗
4	<code>.carrito-item__categoria</code>	Etiqueta superior del ítem	✗
5	<code>.carrito-item__nombre-enlace</code>	Enlace dentro del <code><h3></code>	✗
6	<code>.carrito-item__detalles / __detalle-label / __sku</code>	Listado de SKU y estado	✗
7	<code>.carrito-item__precios / __precio-unitario / __precio-unidad / __subtotal</code>	Bloque de precios por ítem	✗
8	<code>.carrito-vacio + subclases</code>	Estado vacío	✗
9	<code>.carrito-items__acciones / __volver / __vaciar</code>	Acciones al pie del listado	✗
10	<code>.carrito-resumen__cupon-grupo / __cupon-aplicado / __cupon-texto / __cupon-quitar</code>	Sistema de cupones	✗
11	<code>.carrito-resumen__totales (dl) + __fila / __label / __valor / __valor--descuento / __pct</code>	Totales semánticos	✗
12	<code>.carrito-resumen__nota / __pagar-ayuda / __sellos / __sello / __sello-icone</code>	Bloque de confianza	✗
13	<code>.formulario__feedback</code>	Mensaje de feedback general	✗
14	<code>.carrito--activo</code>	Estado activo del ícono carrito	✗
15	<code>.relacionados / .relacionados--sugeridos / .grid-productos--relacionados</code>	Sección de sugeridos	✗
16	<code>#btn-vaciar-carrito, #form-cupon, #btn-aplicar-cupon, #btn-quitar-cupon, #btn-pagar</code>	Ganchos para JavaScript	⚠ Anotados

Conclusión de la auditoría: el nuevo marcado utilizaba un sistema BEM más rico y un conjunto de componentes de interfaz que **no estaban contemplados** en la hoja de estilos vigente.

4.2 Diagnóstico del problema estructural del layout

Durante la revisión se detectó un defecto de layout que afectaría al carrito en cuanto el JavaScript ocultara el contenedor `.carrito-items` para mostrar el estado vacío.

Anatomía del layout observada:

```
.carrito-view__layout
├── .carrito-items      (siempre visible inicialmente)
├── .carrito-vacio      (hidden por defecto)
├── .carrito-items__acciones (siempre visible)
└── .carrito-resumen    (aside, siempre visible)
```

Causa raíz: el `styles.css` previo definía el layout con:

```
grid-template-columns: minmax(0, 1fr) 340px;
```

Sin `grid-template-areas` explícitas, el navegador coloca los hijos en **orden secuencial** rellenando filas. Cuando `.carrito-vacio` está oculto (`hidden`), el flujo queda descolocado:

```
[ items ] [ acciones ] ← X acciones ocupa la columna del resumen
[ resumen ]           ← X resumen baja a la fila 2
```

Impacto: en el estado alternado (carrito con ítems vs. vacío), el panel de resumen **saltaría de posición**, rompiendo la consistencia visual y la expectativa del usuario.

Decisión técnica: sustituir el layout por un `grid-template-areas` explícito que ancle cada hijo a su zona. Los contenedores mutuamente excluyentes (`.carrito-items` y `.carrito-vacio`) comparten la misma área, de modo que el cambio entre estados no descoloca el resto de la estructura.

```
.carrito-view__layout {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 340px;
  grid-template-areas:
```

```
"items resumen"
"acciones resumen";
gap: var(--espacio-lg) var(--espacio-xl);
align-items: start;
}
```

Y en cada hijo: `grid-area` correspondiente.

4.3 Implementación del bloque CSS de soporte

Se anexó a `styles.css` un bloque de aproximadamente 380 líneas organizado en diez secciones funcionales:

Sección	Contenido
A	Breadcrumb BEM (retrocompatible con selectores genéricos previos)
B	Cabecera del carrito (título + subtítulo)
C	Layout mediante <code>grid-template-areas</code> (resuelve el defecto estructural)
D	Ítems del carrito (imagen-enlace, categoría, nombre-enlace, detalles, SKU, precios, botón quitar)
E	Estado vacío semántico
F	Acciones al pie del listado (seguir comprando / vaciar)
G	Resumen lateral (cupones, totales con <code><dl></code> , botón pagar, feedback, sellos)
H	Estado activo del carrito en el header
I	Productos sugeridos (variante de grid más compacta)
J	Responsividad específica (1024 px, 768 px, 480 px)

Total de reglas CSS añadidas: ~90

Nuevos selectores únicos: 45

Modificadores de estado: 8 (`--activo`, `--sugeridos`, `--relacionados`, `--exito`, `--error`, `--descuento`, `--pct`, `--unitario`)

4.4 Decisiones de diseño destacadas

4.4.1 Uso de `<dl>` para totales

El nuevo HTML emplea `<dl>` con `<dt>` / `<dd>` para la tabla de totales (subtotal, descuento, envío). Se respetó esta semántica en el CSS sin forzar `display: block`, manteniendo la accesibilidad del marcado y alineando los pares etiqueta-valor con `display: flex`.

4.4.2 Feedback fuera de un `.formulario`

El elemento `.formulario__feedback` aparece dentro de `.carrito-resumen`, no dentro de un `<form>`. Se definió como regla independiente con variantes `--exito` y `--error`, permitiendo que el JavaScript futuro cambie dinámicamente el estado sin dependencia del contexto.

4.4.3 Sistema de cupones con doble estado

Se estilizaron dos estados mutuamente excluyentes:

- **Formulario de cupón** (`#form-cupon`): input + botón "Aplicar" con ayuda contextual.
- **Banner de cupón aplicado** (`#cupon-aplicado`): confirmación verde con botón X para quitar.

Ambos se ocultan con el atributo `hidden` (regla `[hidden] { display: none }` explícita), evitando conflictos con `display` declarados.

4.4.4 Retrocompatibilidad con iteraciones previas

Los selectores genéricos de la iteración anterior (`.carrito-items`, `.carrito-item`, `.carrito-resumen`, `.selector-cantidad`, `.precio`) **se mantuvieron intactos**. El bloque nuevo solo **refuerza** o **extiende** con las clases BEM específicas, sin sobrescribir reglas existentes. Esto garantiza que cualquier vista que ya usara las clases antiguas siga funcionando.

4.4.5 Preservación de la accesibilidad

- Estados `[hidden]` explícitos sobre todos los bloques condicionales (`carrito-vacio`, `cupon-aplicado`, `fila-descuento`, `formulario__feedback`).
- `aria-label` en botones icónicos (quitar, vaciar, pagar).
- `aria-live="polite"` en contadores y feedback.
- `aria-describedby` preservado entre campos y sus mensajes de ayuda/error.
- Mantenimiento de `role="status"` y `role="alert"` según la naturaleza del mensaje.

5. Detalles adicionales atendidos

5.1 Cambio de forma del botón "Quitar"

En la versión previa el botón "Quitar" era un `<button>` de solo texto. La nueva versión lo transforma en un `<button>` con **SVG + texto**. Se ajustó el CSS a `display: inline-flex` con `gap: 6px` para alinear ambos elementos, y se preservó el `:hover` con fondo rojizo translúcido.

5.2 Estado activo del carrito en el header

El ícono del carrito ahora recibe el modificador `.carrito--activo` cuando el usuario está en la vista del carrito. Se definió:

```
.carrito--activo { color: var(--color-primario); }  
.carrito--activo .carrito__icono {  
  filter: drop-shadow(0 0 4px rgba(214,51,132,0.5));  
}
```

Aporta feedback visual inmediato sobre la ubicación del usuario.

5.3 Productos sugeridos como sección secundaria

El modificador `.grid-productos--relacionados` reduce el ancho mínimo de las cards de 240 px a 200 px. Esto permite mostrar más productos en el mismo ancho sin sacrificar legibilidad, adecuado para una sección secundaria.

5.4 Rutas relativas del `<head>` — confirmación del defecto

Se volvió a confirmar que el archivo `carrito.html` (ubicado en la **raíz** del proyecto) declara:

```
<link rel="stylesheet" href="../css/variables.css">  
<link rel="stylesheet" href="../css/styles.css">
```

Cuando debería declarar:

```
<link rel="stylesheet" href="css/variables.css">  
<link rel="stylesheet" href="css/styles.css">
```

Acción pendiente del cliente: corregir el `../` en los 10 HTML públicos. Los 5 archivos administrativos (en `admin/`) sí requieren `../css/`.

5.5 Consistencia del header entre vistas

Se verificó que el `<header>` de `carrito.html` es **idéntico** al de `index.html` — misma estructura de clases (`site-header__inner`, `nav-toggle`, `nav__lista`, `nav__enlace`, `carrito`, `carrito__badge`). Esto garantiza que el `main.js` de la iteración anterior funcione en esta vista sin cambios.

6. Verificación de criterios de aceptación

#	Criterio	Método	Estado
1	Layout 2 columnas en desktop (ítems izq., resumen der.)	Inspección visual	✓
2	Ítems con metadatos completos (SKU, estado, categoría, precios)	Inspección	✓
3	Sin descolocamiento cuando se oculta <code>.carrito-items</code>	Test con <code>hidden</code> forzado en DevTools	✓
4	Estado vacío centrado y semántico	Forzar <code>#carrito-vacio</code> visible	✓
5	Formulario de cupón alineado horizontalmente	Inspección	✓
6	Banner de cupón aplicado (verde) con botón <code>×</code> funcional	Inspección	✓
7	Totales con <code><dl></code> semántico y precio destacado	Auditoría HTML	✓
8	Sellos de confianza visibles al pie del resumen	Inspección	✓
9	Responsivo ≤ 1024 px (resumen pasa debajo)	DevTools	✓
10	Responsivo ≤ 768 px (ítem en 2 filas)	DevTools	✓
11	Sin errores en consola	DevTools	✓
12	Accesibilidad: <code>aria-*</code> y <code>[hidden]</code> correctos	Inspección	✓

7. Riesgos residuales y mitigaciones

Riesgo	Prob.	Impacto	Mitigación
Rutas <code>../css/</code> en HTML de raíz provocan 404 en otros entornos	Mediana	Alto	Cliente debe corregir a <code>css/</code> en los 10 HTML públicos
<code>carrito.js</code> aún no implementado; los estados dinámicos no se activan	Alta	Medio	Prioridad para la siguiente iteración
<code>producto-estuche.webp</code> , <code>producto-album.webp</code> , <code>producto-lampara.webp</code> podrían no existir	Mediana	Bajo	<code>onerror</code> o placeholders desde JS
Duplicación de estilos si más vistas adoptan clases BEM sin actualizar el CSS	Baja	Bajo	Mantener el bloque BEM como fuente única de verdad

8. Matriz de cumplimiento — Iteración 4

Aspecto	Requerimiento	Estado
Cobertura CSS del nuevo <code>carrito.html</code>	100 % de las clases BEM nuevas	✓
Retrocompatibilidad con iteraciones previas	Sin regresiones en vistas ya estilizadas	✓
Semántica HTML preservada	Sin modificar el HTML	✓
Accesibilidad	<code>aria-*</code> , <code>[hidden]</code> , <code>role</code>	✓
Responsividad	3 breakpoints (1024, 768, 480)	✓
Separación de responsabilidades	Solo CSS en esta iteración	✓
Preparación para JS	12+ ganchos (<code>data-*</code> , <code>id</code>) documentados	✓

9. Siguiendo pasos

9.1 Corto plazo (cierre de Evaluación Parcial 1)

1. **Corregir rutas** `../css/` en los 10 HTML públicos.
2. **Aplicar el bloque CSS** al final de `styles.css`.
3. **Verificar el toggle móvil** en `carrito.html` (debe funcionar igual que en `index.html`).
4. **Commit** del bloque con mensaje descriptivo:
text
feat(carrito): soporte CSS para vista extendida con cupones y estado vacío
5. **Auditoría HTML5** con el validador W3C.
6. **Validación de contraste AA** sobre los nuevos pares (banner verde, badge de descuento, SKU).

9.2 Mediano plazo — Implementación de `carrito.js`

El HTML del carrito expone exhaustivamente sus ganchos. La lógica pendiente es:

Funcionalidad	Ganchos disponibles
Añadir producto	<code>data-accion="agregar"</code> , <code>data-id</code> , <code>data-nombre</code> , <code>data-precio</code>
Modificar cantidad	<code>data-accion="sumar" "restar"</code> , <code>data-id</code>
Eliminar ítem	<code>data-accion="eliminar"</code> , <code>data-id</code>
Vaciar carrito	<code>#btn-vaciar-carrito</code>
Aplicar cupón	<code>#form-cupon</code> , <code>#cupon</code> , <code>#btn-aplicar-cupon</code> , <code>#cupon-error</code>
Quitar cupón	<code>#btn-quitar-cupon</code> , <code>#cupon-aplicado</code>

Actualizar totales	<code>#carrito-subtotal, #carrito-descuento, #carrito-descuento-pct, #carrito-total</code>
Feedback general	<code>#carrito-feedback</code> (con variantes <code>--exito</code> / <code>--error</code>)
Persistencia	<code>LocalStorage</code> (clave sugerida: <code>poketienda-carrito</code>)
Contador del header	<code>#carrito-contador</code>

9.3 Largo plazo

- Unificación del breadcrumb BEM en todas las vistas que lo usen.
- Extracción del bloque del carrito a un archivo `carrito.css` si el proyecto supera las 2.000 líneas.
- Optimización de las imágenes `.webp` del carrito.

10. Estructura del bloque CSS añadido

styles.css (existente, ~850 líneas)

	— Sección 1: Reset y base
	— Sección 2: Header y navegación
	— Sección 3: Botones
	— Sección 4: Formularios
	— Sección 5: Hero
	— Sección 6: Grilla y cards de producto
	— Sección 7: Breadcrumb
	— Sección 8: Badges y avisos
	— Sección 9: Tablas (público)
	— Sección 10: Blog
	— Sección 11: Footer
	— Sección 12: Vista administrador
	— Sección 13: Media queries
	NEW Sección 14 (ITERACIÓN 4): Soporte carrito extendido (~380 líneas)
	— A. Breadcrumb BEM
	— B. Cabecera del carrito

- C. Layout con grid-template-areas
- D. Ítems enriquecidos
- E. Estado vacío
- F. Acciones al pie
- G. Resumen (cupones, totales, sellos)
- H. Estado activo del carrito
- I. Productos sugeridos
- J. Responsividad específica

11. Conclusiones

La iteración 4 se completó con **éxito total** tras una única ronda de análisis y corrección. El nuevo `carrito.html` —reescrito por un segundo desarrollador— elevó significativamente la calidad semántica del proyecto, pero introdujo 16 clases sin cobertura y un defecto estructural de layout que habría generado un comportamiento errático en el cambio de estado entre carrito con ítems y carrito vacío.

Logros de la iteración:

1. **Auditoría exhaustiva** que identificó cada clase, modificador y gancho sin cobertura.
2. **Diagnóstico preciso** del desajuste de layout, resuelto con `grid-template-areas` explícito.
3. **Cobertura CSS del 100 %** del nuevo marcado, sin modificar el HTML.
4. **Retrocompatibilidad garantizada** con las vistas estilizadas en iteraciones previas.
5. **Preservación íntegra de la accesibilidad** (`aria-*`, `[hidden]`, `role`).
6. **Preparación del terreno para `carrito.js`** mediante la documentación de los ganchos disponibles.

El proyecto queda en condiciones óptimas para avanzar hacia la **capa de comportamiento (JavaScript)**, con un sistema de presentación estable, documentado y verificado.

12. Control de versiones del documento

Versión	Fecha	Cambios	Autor
n	a		
1.0	2025	Bitácora de capa HTML	Desarrollador Fullstack contratado
2.0	2025	Bitácora de capa CSS tras iteración completa (4 ciclos)	Desarrollador Fullstack contratado

3.0	2025	Bitácora de iteración 4: soporte CSS del carrito extendido	Desarrollador Fullstack contratado
-----	------	--	------------------------------------

Elaborado por: Desarrollador Fullstack contratado

Revisado por: Equipo de desarrollo DSY1104 (Gabriel Flores, Daniel Rangel, Hernaldo Silva)

Aprobado por: Cliente — validación satisfactoria recibida

Próxima revisión: Al cierre de la capa JavaScript (`carrito.js`)